Tree-Type Networks of Small Neural Networks

Graded assignment: tree type network revisited — CIFAR-10 and Cats vs Dogs

Ronak Raichandani (2026EEY7568)

Code (private repository): https://github.com/ronakraichandani/MLOAssignment_2026EEY7568_Ronak

1. Architecture and implementation

Starting point. In the original tree-type network a single neuron is trained first; the samples it misclassifies are partitioned into false negatives and false positives, and two child neurons (A and B) are trained to fire exactly on those samples and to stay silent on the others, with *don't-care* outputs on samples that are already safe. The children feed the parent through a positive (A) and a negative (B) weight, so every training sample ends up correctly classified. If a child's problem is not linearly separable, it grows children of its own.

What is new here. (i) Every node is a *small neural network* (linear, or a one-hidden-layer MLP) operating on features from a frozen trunk. (ii) The construction is extended to **multiclass**: the root has K logits; for each class k the child A_k fires on the class-k samples the root did not predict as k (positive set), is silent on all samples whose label is not k, and is don't-care on true positives of class k. The corrected logit is $z'_k = z_k + w_k \cdot \text{step}(A_k(x))$. B children are redundant for K≥2 (raising the true logit already repairs every error), which halves the number of children; they are used for binary (child) nodes. (iii) The correction weights are set in **closed form** instead of by inspection: w_k is the largest logit gap over the class-k false negatives plus a margin. (iv) Each node searches the **smallest hidden width** that interpolates its own masked training set (0 = linear, then 16, 64, 256 by default), so nodes are only as large as needed.

Guarantee. If every node interpolates its own masked problem, the whole tree classifies the training set perfectly: an error sample is a positive of exactly the child that raises its true logit by more than the gap; samples outside the positive set cannot be disturbed because every child is silent on samples whose label differs from the class it repairs, and may only raise the true logit on its don't-care set. Recursion terminates because masked problems shrink. This is verified by unit tests (tests/test_tree_logic.py) on noisy synthetic and noisy digit data.

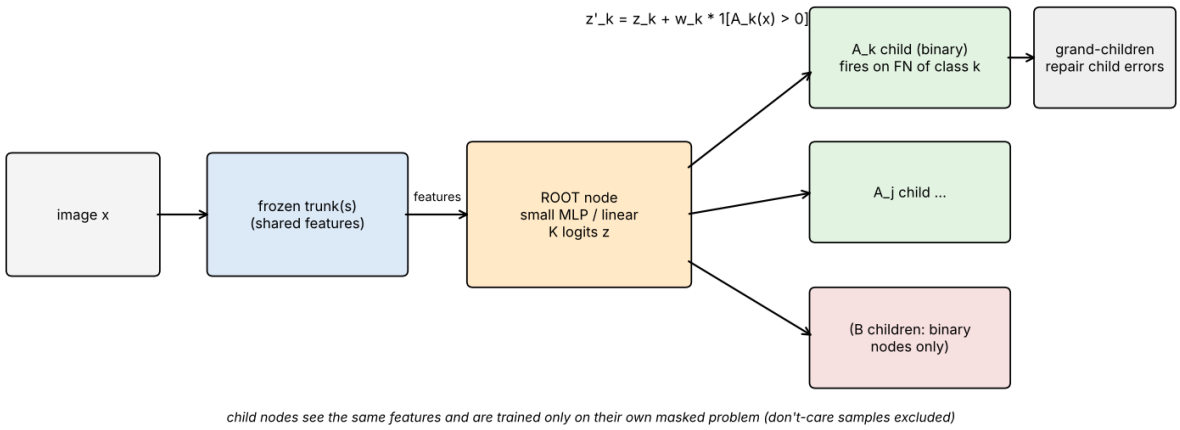


Figure 1: Tree architecture. Nodes share frozen trunk features; each child solves a masked binary problem and is added to the logit of the class it repairs.

Features. The task asks how to provide appropriate features. Implemented options (features.py): a *scratch CNN trunk* trained for only a few epochs on the (noisy) training labels and then frozen (so that it solves part of the problem and leaves a well-defined remainder for the tree); a frozen *random CNN*; an ImageNet-pretrained *ResNet-18* (uses external data, so reported separately); and *multi-trunk* concatenations of separately trained trunks. All nodes of a tree share the trunk, which also makes every node differentiable down to the pixels for receptive-field analysis.

Implementation. Python / PyTorch. tree.py contains the growth logic (backend independent), learners_torch.py the node networks, rf.py the receptive-field tools; scripts/ holds one script per experiment. Hyper-parameters of every run are stored in its results.json and listed below.

2. Results: interpolation on CIFAR-10 and Cats vs Dogs

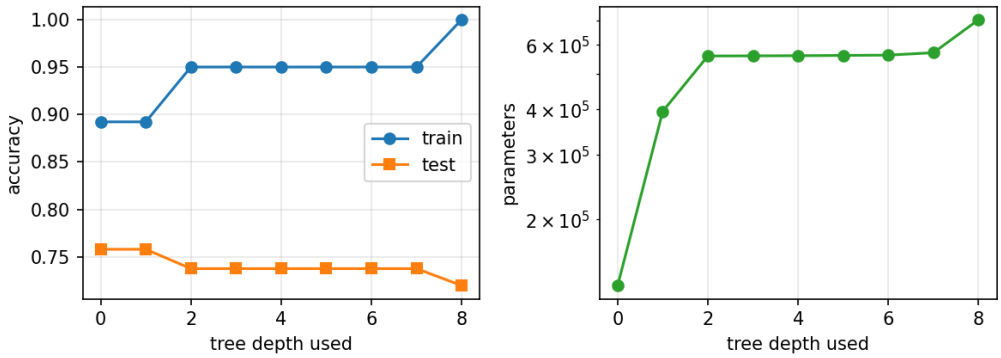
2.1 Run *catsdogs_clean* (catsdogs)

trunk: scratch_cnn (epochs 3), node widths tried: 0,16,64,256, max depth 8, label noise 0.0, train/test size 19998/5000, feature dim 512, seed 0.

	train acc	test acc
root node only	89.22%	75.80%
full tree	99.99%	71.98%

nodes	15
parameters	703436
max depth	8
nodes per depth	{'0': 1, '1': 2, '2': 2, '3': 1, '4': 1, '5': 2, '6': 2, '7': 2, '8': 2}
node width histogram	{'0': 8, '16': 1, '64': 1, '256': 5}
train accuracy w.r.t. clean labels	99.99%
noisy labels memorised	n/a
growth time (s)	165

depth used	train acc	test acc	params
0	89.22%	75.80%	131846
1	89.22%	75.80%	395020
2	95.00%	73.76%	559504
3	95.00%	73.76%	560019
4	95.00%	73.76%	560536
5	95.00%	73.76%	561566
6	95.00%	73.76%	562596
7	95.00%	73.76%	571338
8	99.99%	71.98%	703436



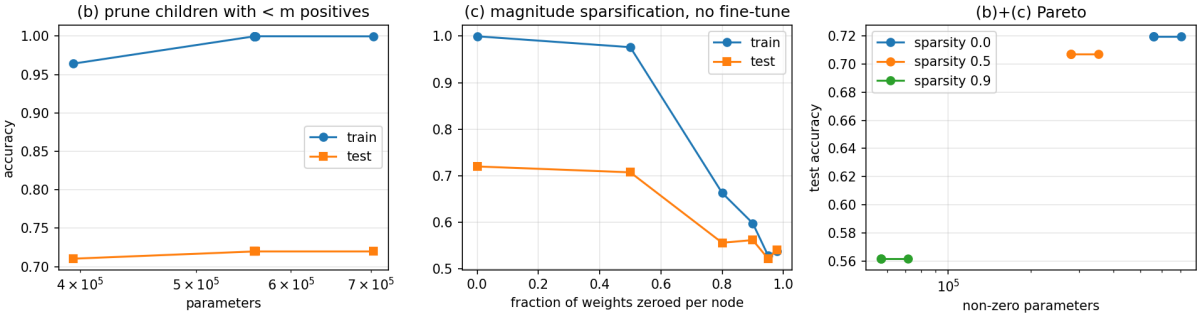
Accuracy and size when only complete sub-trees up to a given depth are used.

Size minimisation (pruning children that repair fewer than m samples, magnitude sparsification, no fine-tuning):

m	params	nodes	train acc	test acc
1	703436	15	99.96%	71.98%
2	560532	7	99.97%	71.98%

m	params	nodes	train acc	test acc
3	559502	5	99.97%	71.98%
5	559502	5	99.97%	71.98%
10	559502	5	99.97%	71.98%
20	559502	5	99.97%	71.98%
50	559502	5	99.97%	71.98%
100	559502	5	99.97%	71.98%
200	559502	5	99.97%	71.98%
500	395016	3	96.42%	71.04%

sparsity	non-zero params	train acc	test acc
0.0	703436	99.96%	71.98%
0.5	352420	97.62%	70.72%
0.8	141818	66.30%	55.58%
0.9	71616	59.77%	56.16%
0.95	36513	52.83%	52.16%
0.98	15454	53.72%	54.10%



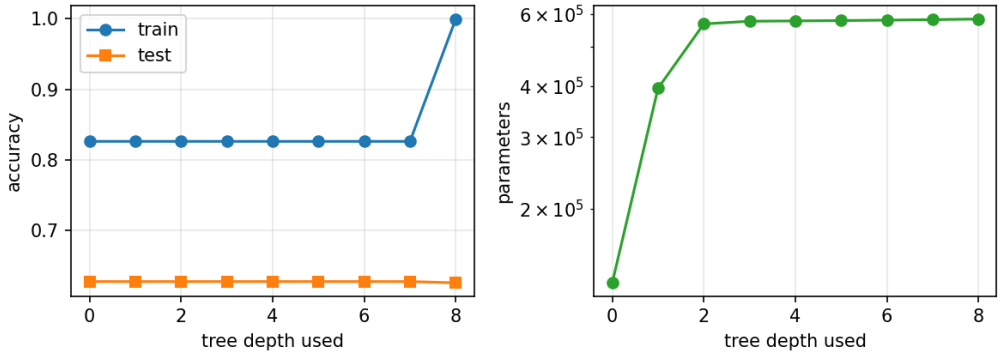
2.2 Run *catsdogs_noise15* (catsdogs)

trunk: scratch_cnn (epochs 3), node widths tried: 0,16,64,256, max depth 8, label noise 0.15, train/test size 19998/5000, feature dim 512, seed 0.

	train acc	test acc
root node only	82.64%	62.80%
full tree	99.95%	62.62%

nodes	22
parameters	583681
max depth	8
nodes per depth	{'0': 1, '1': 2, '2': 3, '3': 2, '4': 2, '5': 2, '6': 3, '7': 3, '8': 4}
node width histogram	{'0': 15, '16': 2, '64': 1, '256': 4}
train accuracy w.r.t. clean labels	84.99%
noisy labels memorised	99.90%
growth time (s)	232

depth used	train acc	test acc	params
0	82.64%	62.80%	131846
1	82.64%	62.80%	395022
2	82.64%	62.80%	567733
3	82.64%	62.80%	576475
4	82.64%	62.80%	577505
5	82.64%	62.80%	578537
6	82.64%	62.80%	580082
7	82.64%	62.80%	581629
8	99.95%	62.62%	583681



Accuracy and size when only complete sub-trees up to a given depth are used.

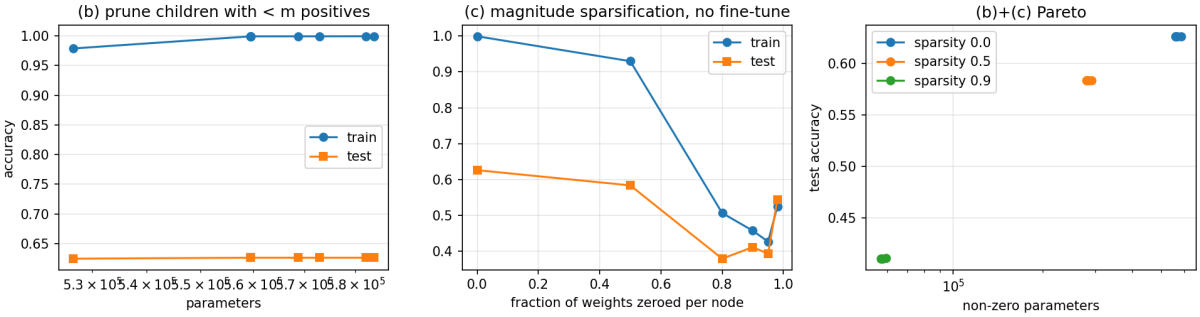
Repair depth as a label-noise detector. The depth of the deepest repair node that fires on a training sample separates mislabelled from clean samples with AUROC 0.560. Fraction of flipped labels per repair depth: {'0': 0.13, '1': 0.214, '2': 0.143}.

Size minimisation (pruning children that repair fewer than m samples, magnitude sparsification, no fine-tuning):

m	params	nodes	train acc	test acc
1	583681	22	99.93%	62.60%
2	582136	19	99.93%	62.60%
3	572879	16	99.92%	62.60%
5	568759	8	99.92%	62.60%
10	559502	5	99.92%	62.60%
20	559502	5	99.92%	62.60%
50	559502	5	99.92%	62.60%
100	559502	5	99.92%	62.60%
200	559502	5	99.92%	62.60%
500	526603	4	97.88%	62.44%

sparsity	non-zero params	train acc	test acc
0.0	583681	99.93%	62.60%
0.5	292433	92.99%	58.36%
0.8	117696	50.65%	37.92%
0.9	59449	45.71%	41.12%
0.95	30320	42.66%	39.28%

sparsity	non-zero params	train acc	test acc
0.98	12850	52.43%	54.30%



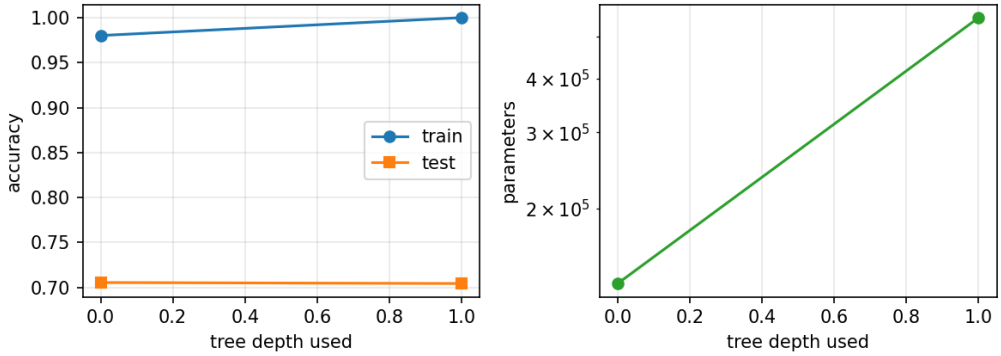
2.3 Run *cifar10_clean* (cifar10)

trunk: scratch_cnn (epochs 3), node widths tried: 0,16,64,256, max depth 8, label noise 0.0, train/test size 50000/10000, feature dim 512, seed 0.

	train acc	test acc
root node only	98.01%	70.54%
full tree	100.00%	70.43%

nodes	10
parameters	553349
max depth	1
nodes per depth	{'0': 1, '1': 9}
node width histogram	{'16': 3, '64': 4, '256': 3}
train accuracy w.r.t. clean labels	100.00%
noisy labels memorised	n/a
growth time (s)	155

depth used	train acc	test acc	params
0	98.01%	70.54%	133916
1	100.00%	70.43%	553349



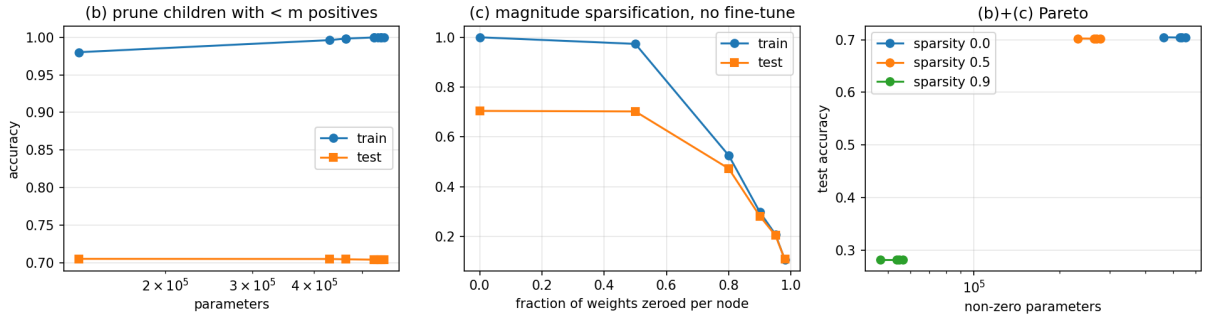
Accuracy and size when only complete sub-trees up to a given depth are used.

Size minimisation (pruning children that repair fewer than m samples, magnitude sparsification, no fine-tuning):

m	params	nodes	train acc	test acc
1	553349	10	99.99%	70.43%

m	params	nodes	train acc	test acc
2	545122	9	99.99%	70.43%
3	545122	9	99.99%	70.43%
5	536895	8	99.98%	70.43%
10	528668	7	99.97%	70.43%
20	528668	7	99.97%	70.43%
50	462870	5	99.82%	70.48%
100	462870	5	99.82%	70.48%
200	429971	4	99.62%	70.52%
500	133898	1	98.01%	70.54%

sparsity	non-zero params	train acc	test acc
0.0	553349	99.99%	70.43%
0.5	277229	97.34%	70.22%
0.8	111562	52.53%	47.28%
0.9	56338	29.78%	28.16%
0.95	28726	20.84%	20.45%
0.98	12159	10.70%	10.85%



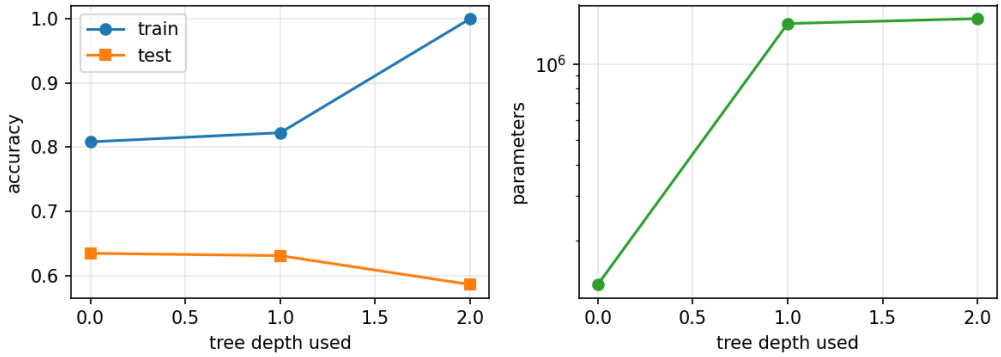
2.4 Run *cifar10_noise15* (cifar10)

trunk: scratch_cnn (epochs 3), node widths tried: 0,16,64,256, max depth 8, label noise 0.15, train/test size 50000/10000, feature dim 512, seed 0.

	train acc	test acc
root node only	80.83%	63.49%
full tree	100.00%	58.63%

nodes	20
parameters	1516099
max depth	2
nodes per depth	{'0': 1, '1': 10, '2': 9}
node width histogram	{'0': 1, '16': 8, '256': 11}
train accuracy w.r.t. clean labels	84.82%
noisy labels memorised	100.00%
growth time (s)	353

depth used	train acc	test acc	params
0	80.83%	63.49%	133918
1	82.25%	63.12%	1449786
2	100.00%	58.63%	1516099



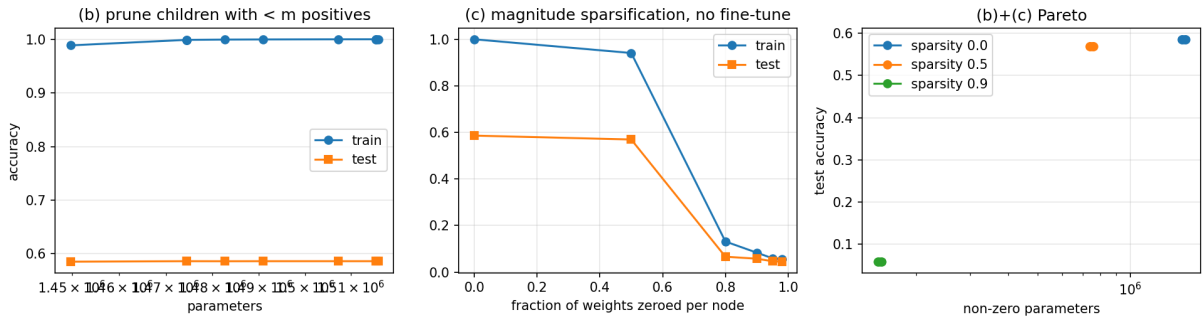
Accuracy and size when only complete sub-trees up to a given depth are used.

Repair depth as a label-noise detector. The depth of the deepest repair node that fires on a training sample separates mislabelled from clean samples with AUROC 0.754. Fraction of flipped labels per repair depth: {'0': 0.068, '1': 0.453, '2': 0.0}.

Size minimisation (pruning children that repair fewer than m samples, magnitude sparsification, no fine-tuning):

m	params	nodes	train acc	test acc
1	1516099	20	99.99%	58.62%
2	1515584	19	99.99%	58.62%
3	1515584	19	99.99%	58.62%
5	1507357	18	99.98%	58.62%
10	1490903	16	99.95%	58.62%
20	1482676	15	99.93%	58.61%
50	1474449	14	99.88%	58.62%
100	1474449	14	99.88%	58.62%
200	1474449	14	99.88%	58.62%
500	1449768	11	98.85%	58.53%

sparsity	non-zero params	train acc	test acc
0.0	1516099	99.99%	58.62%
0.5	759555	94.09%	56.95%
0.8	305637	13.15%	6.61%
0.9	154325	8.35%	5.77%
0.95	78677	5.95%	4.66%
0.98	33285	5.66%	4.49%



3. Overfitting and double descent

Model-wise / epoch-wise: ResNet-18 of width k on cifar10 (15000 training images, 15% label noise, Adam lr 0.0001, 120 epochs, no augmentation).

k	params	train err (median last 3 evals)	test err (median last 3 evals)	best-epoch test err
1	2991	0.684	0.689	0.687
2	11420	0.513	0.665	0.646
4	44622	0.091	0.689	0.605
6	99616	0.001	0.663	0.573
8	176402	0.000	0.656	0.582
12	395350	0.000	0.625	0.570
16	701466	0.002	0.621	0.582
24	1575202	0.005	0.591	0.572
32	2797610	0.000	0.543	0.536
48	6288442	0.000	0.507	0.501
64	11173962	0.000	0.476	0.473

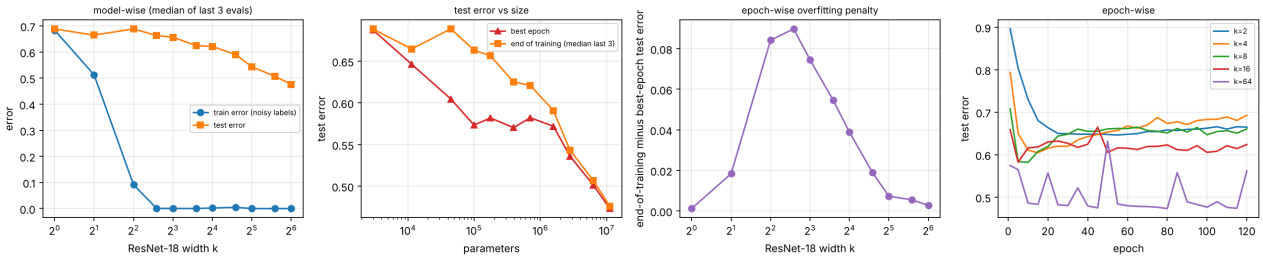
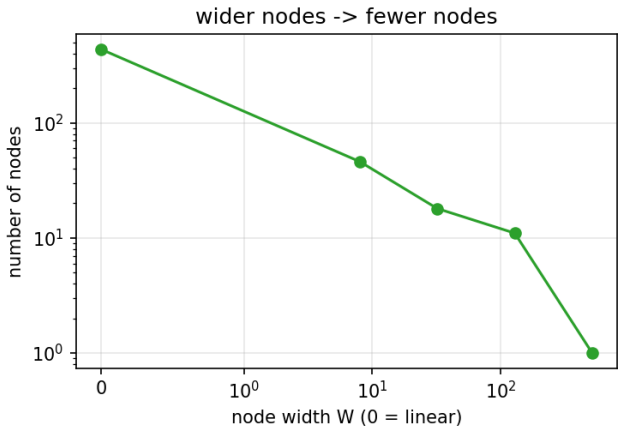


Figure: model-wise (left, middle) and epoch-wise (right) test error.

Tree-wise sweep on cifar10 features (scratch_cnn trunk, 8000 images, 15% label noise): every node has the same fixed hidden width W, trees are grown to interpolation; a single MLP of width W on the same features is the reference.

W	nodes	params	train acc	test acc	root-only test acc
0	437	229670	100.00%	42.12%	54.17%
8	46	189369	100.00%	41.33%	53.28%
32	18	296413	100.00%	42.61%	47.47%
128	11	724904	100.00%	44.45%	44.92%
512	1	267786	100.00%	47.00%	47.00%



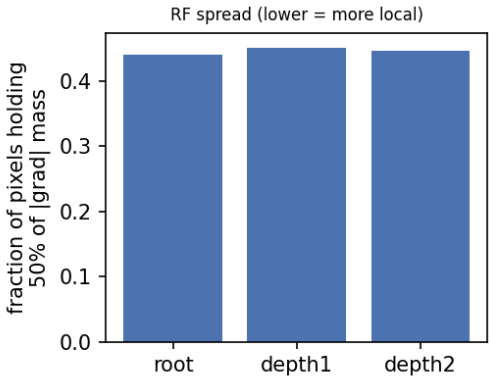


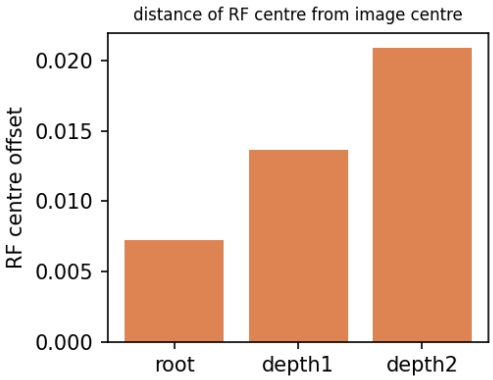
4. Receptive fields of tree nodes

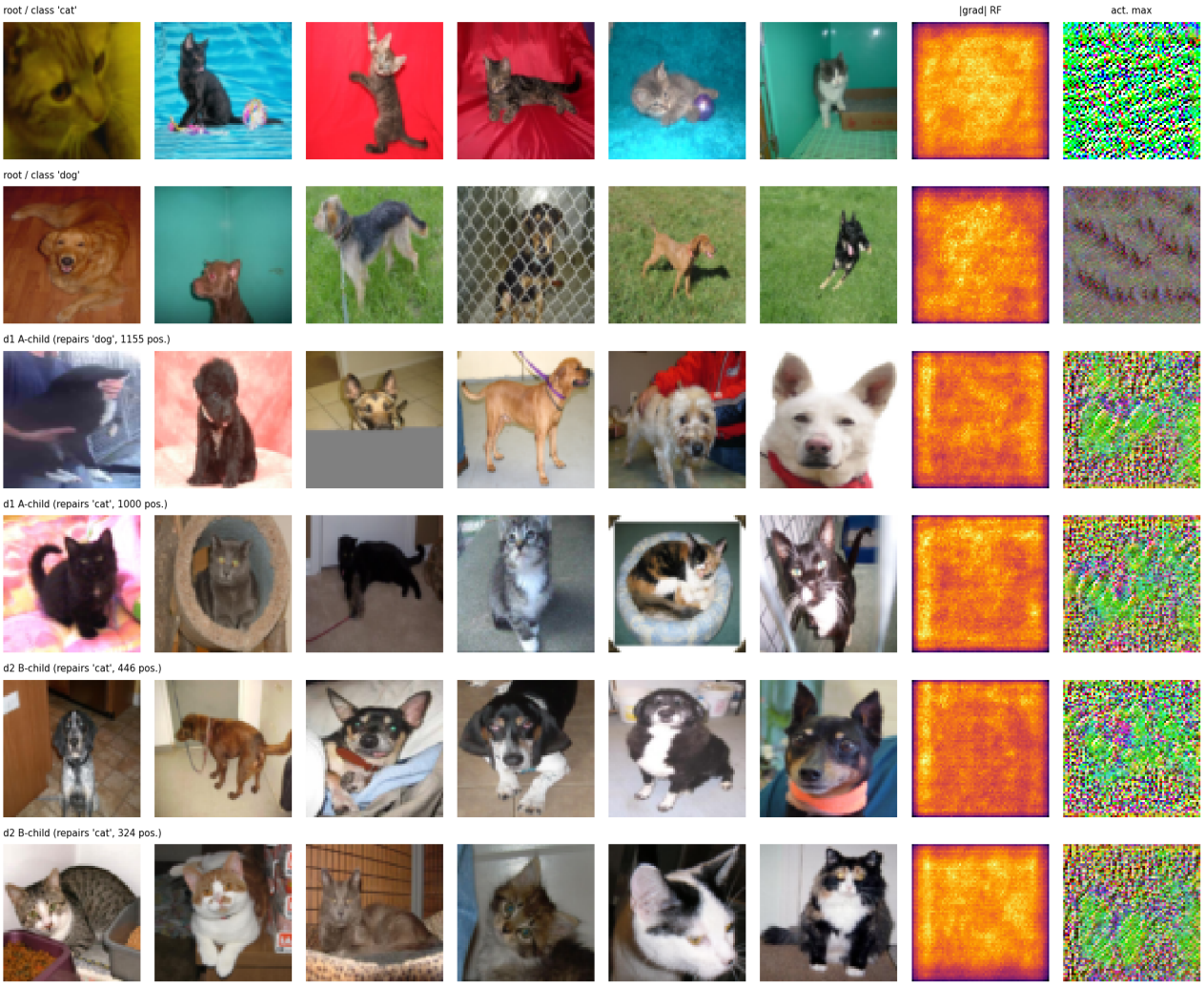
For each node, the gradient receptive field (SmoothGrad |d logit / d pixel| averaged over the node's positive training samples), an activation-maximisation image and the top-activating positives are shown. Localisation statistics are averaged by depth.

Run catsdogs_clean

level	nodes	frac. of pixels for 50% of |grad|	centre offset	normalised entropy
root	2	0.440	0.007	0.998
depth1	2	0.451	0.014	0.999
depth2	2	0.446	0.021	0.998



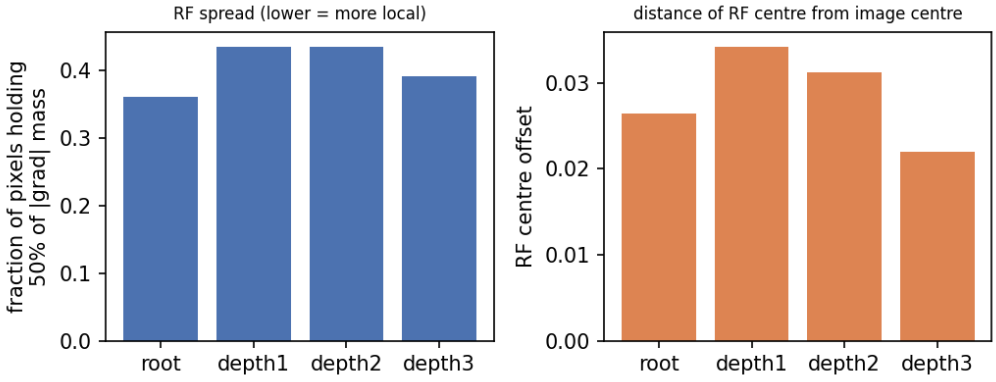


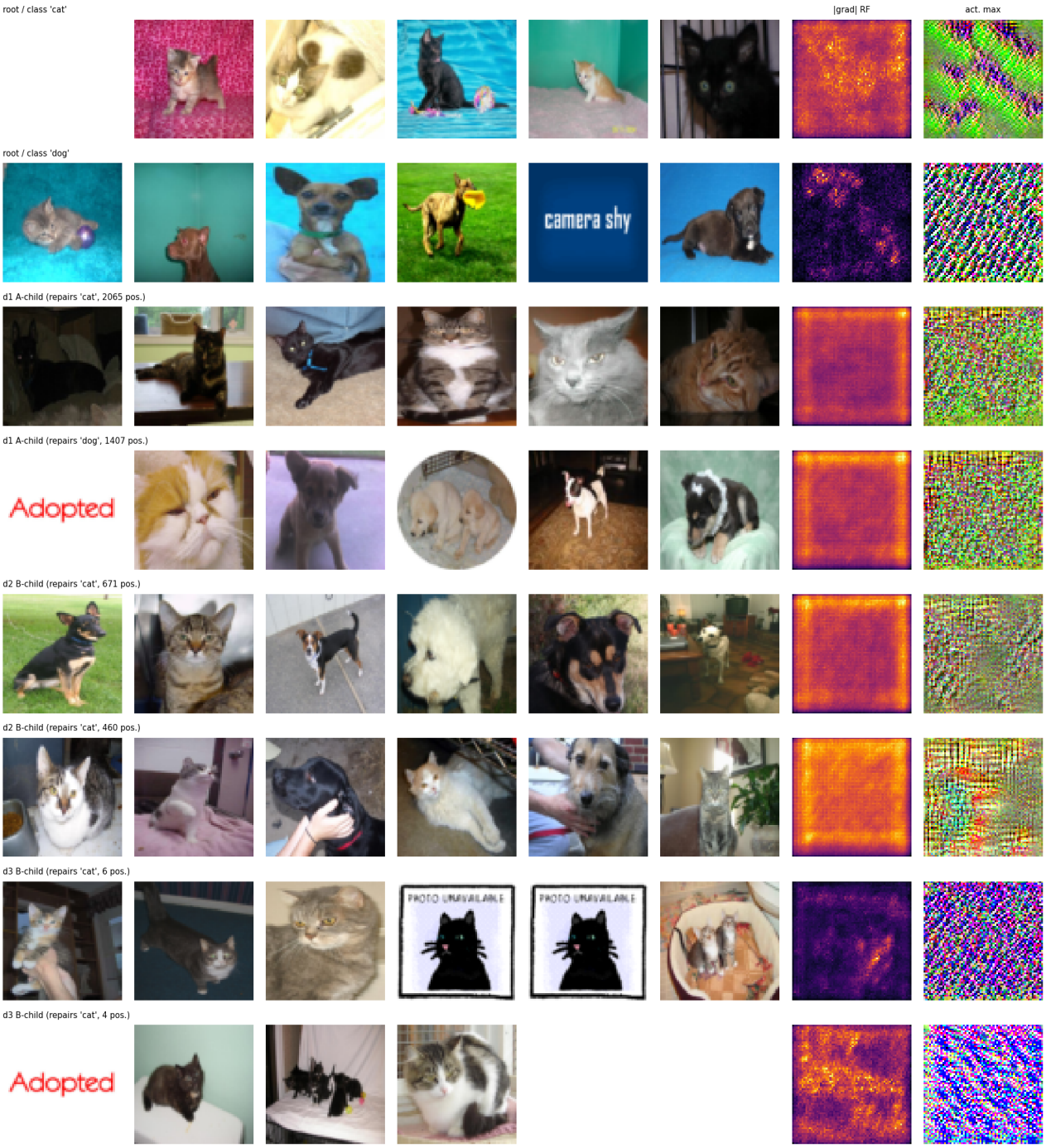


Per node: top-activating positives | gradient RF | activation maximisation.

Run catsdogs_noise15

level	nodes	frac. of pixels for 50% of |grad|	centre offset	normalised entropy
root	2	0.361	0.026	0.989
depth1	2	0.435	0.034	0.998
depth2	2	0.435	0.031	0.998
depth3	2	0.392	0.022	0.995

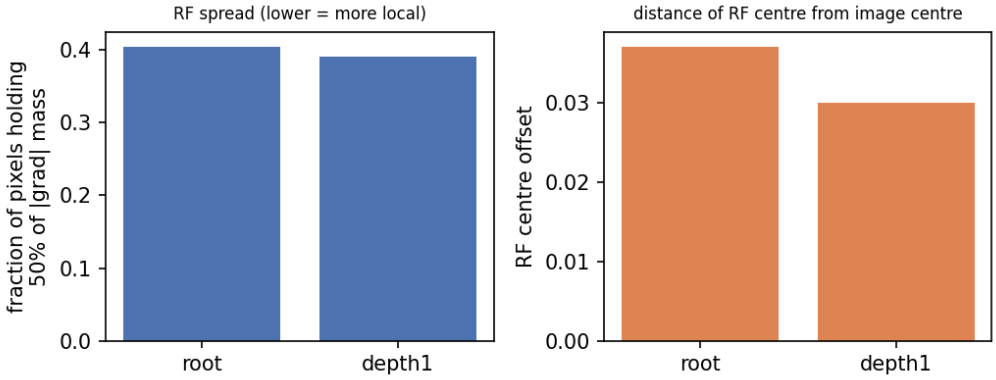


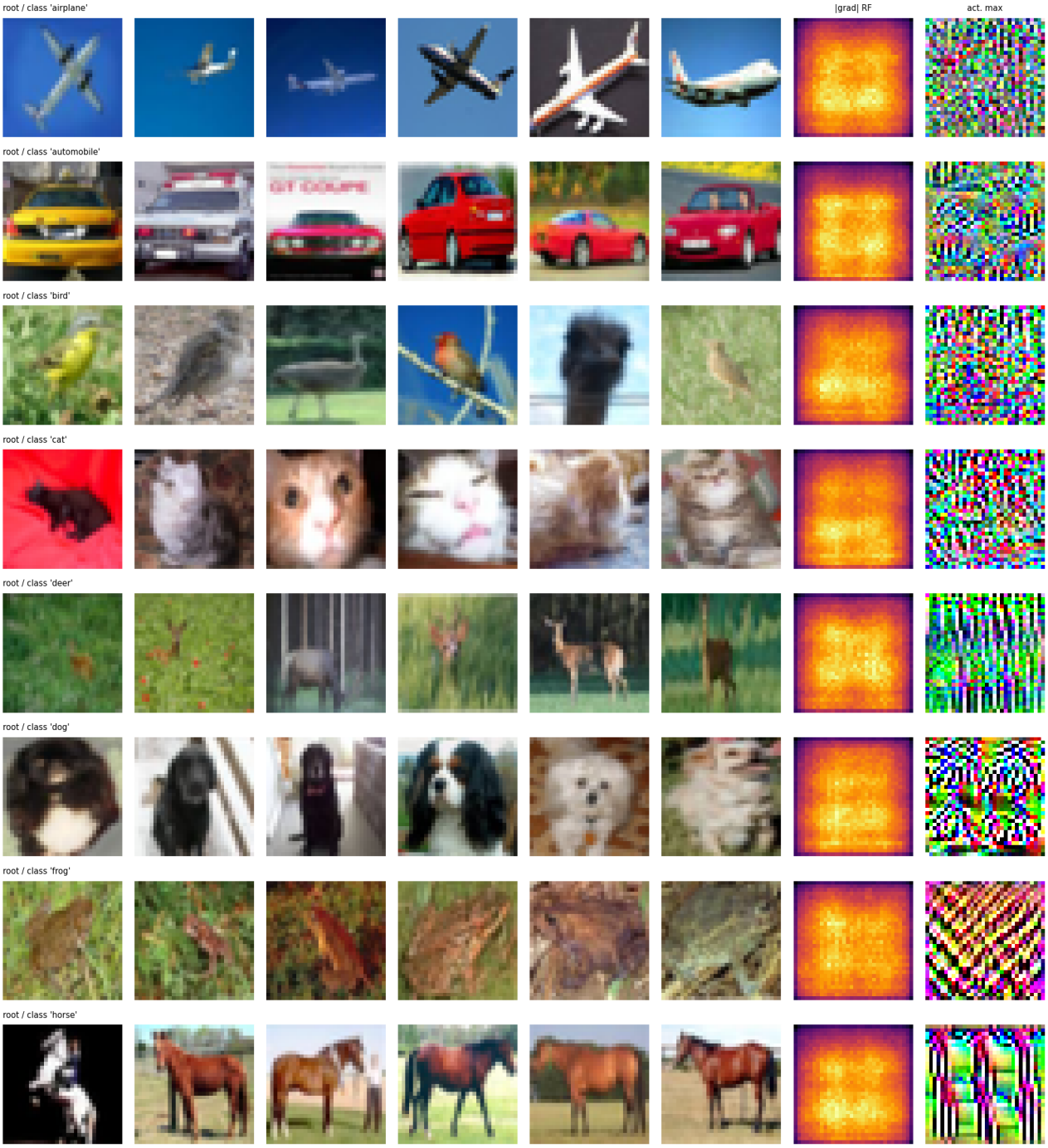


Per node: top-activating positives | gradient RF | activation maximisation.

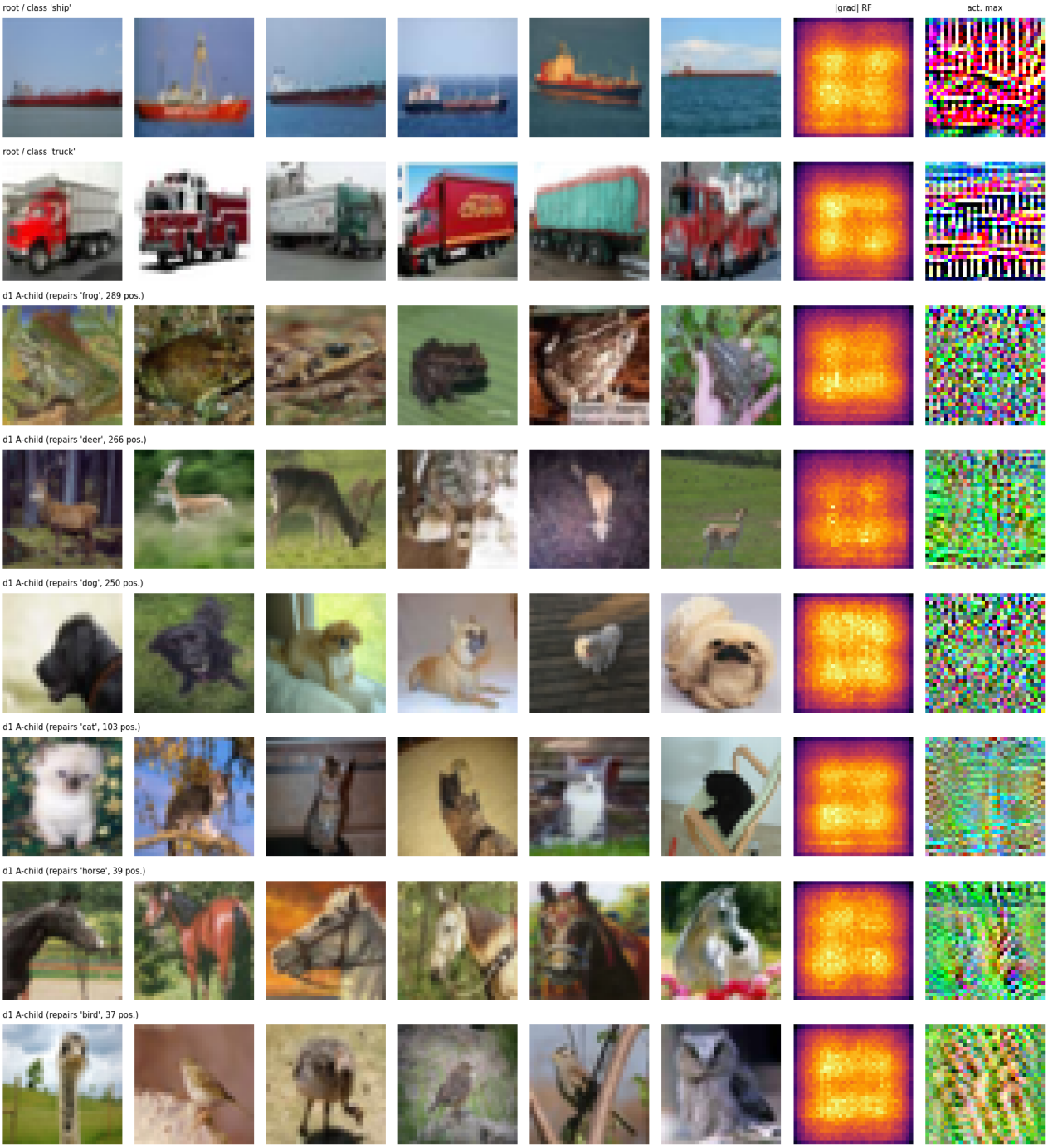
Run cifar10_clean

level	nodes	frac. of pixels for 50% of |grad|	centre offset	normalised entropy
root	10	0.404	0.037	0.995
depth1	8	0.391	0.030	0.994





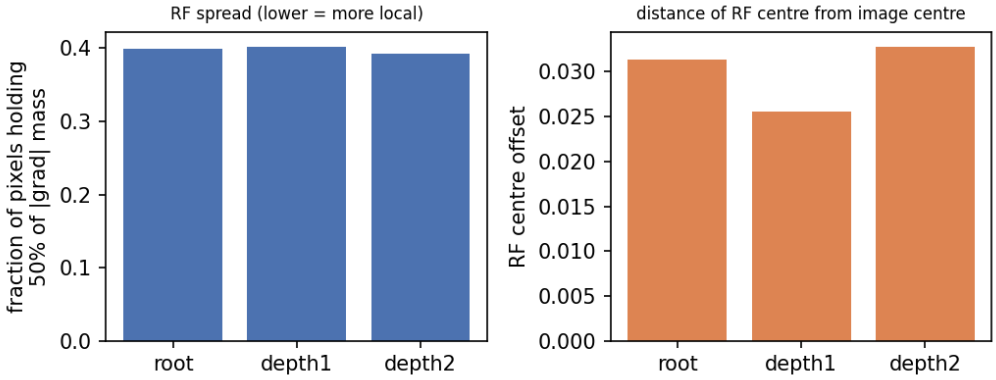
Per node: top-activating positives | gradient RF | activation maximisation.

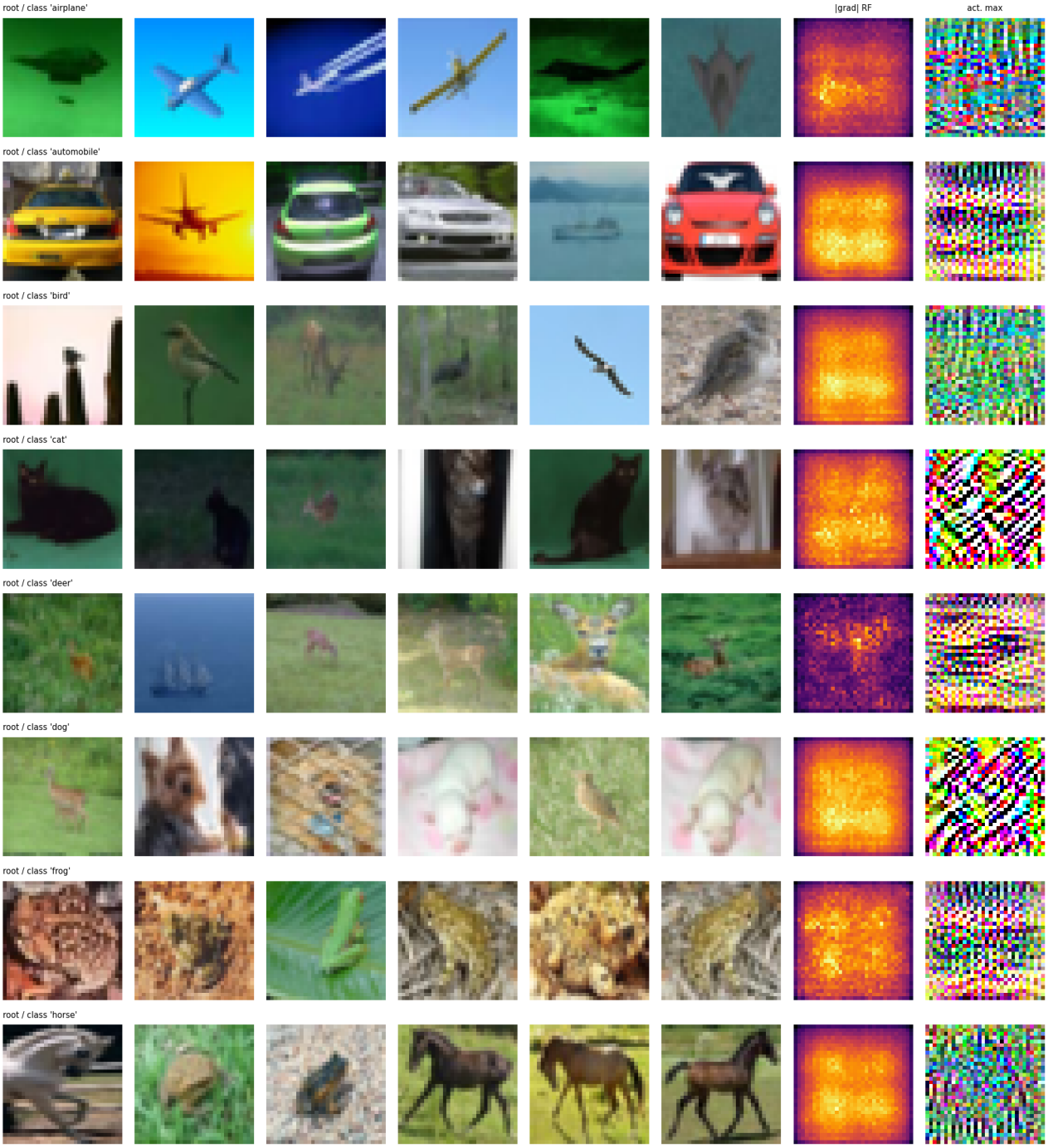


Per node: top-activating positives | gradient RF | activation maximisation.

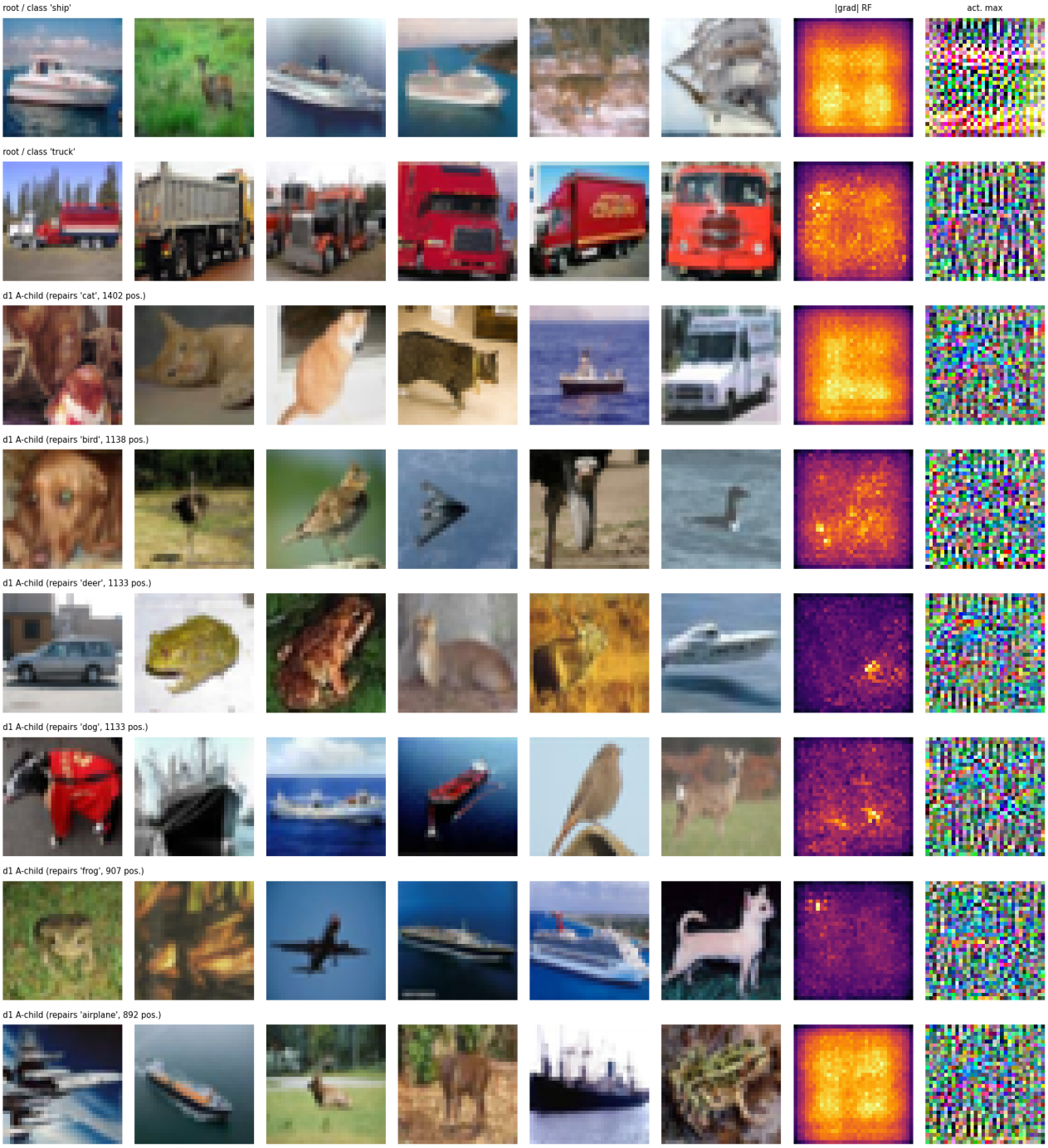
Run cifar10_noise15

level	nodes	frac. of pixels for 50% of |grad|	centre offset	normalised entropy
root	10	0.398	0.031	0.994
depth1	8	0.402	0.026	0.994
depth2	8	0.392	0.033	0.994





Per node: top-activating positives | gradient RF | activation maximisation.



Per node: top-activating positives | gradient RF | activation maximisation.

5. Interpretations and conclusions

Interpolation

The tree reaches exactly 100.0% training accuracy on clean CIFAR-10 (50,000 images) and on CIFAR-10 with 15% flipped labels, and 99.99% / 99.95% on clean / noisy Cats-vs-Dogs (20k images). The two Cats-vs-Dogs trees stopped 1-10 samples short because they hit the max_depth=8 cap, so their guarantee of exact interpolation was cut off by the cap, not by a flaw in the construction.

On clean CIFAR-10 the root already fits 98.0% of the training set, so the tree is tiny (10 nodes, depth 1, 0.55M parameters). With 15% label noise the root fits only 80.8%, and the tree grows to 20 nodes (depth 2, 1.5M parameters) to memorise the flipped labels: it fits 100% of the noisy labels, i.e. 100% of the flipped samples, but then agrees with the true labels on only 84.8% of the training set. Cats-vs-Dogs is the hard case: the root fits 89% (clean) and 83% (noisy), and the trees need 15 and 22 nodes at depth 8, mostly linear (width-0) nodes.

Generalisation and double descent

Model-wise (width-k ResNet-18, 15k CIFAR-10 images, 15% label noise, 120 epochs, single seed), training error reaches about 0 at k=6 (about 100k parameters). Test error at the end of training is 0.689 (k=1), 0.665 (k=2), 0.689 (k=4), 0.663 (k=6), 0.656 (k=8), then falls steadily to 0.625 (k=12), 0.591 (k=24), 0.543 (k=32), 0.507 (k=48) and 0.476 (k=64). So there is only a weak bump at k=4, right where training error leaves the underfitting regime (0.51 to 0.09). I would not call this a clear double-descent peak: the bump is about 2 points and a single seed cannot separate it from noise. What we do see clearly is the second descent: beyond the interpolation threshold, wider is monotonically better, even though every model memorises all the noisy labels.

Epoch-wise, the effect is large and clean. Comparing the final test error with the best test error seen during training, the overfitting penalty is +0.1 points at k=1, +9.0 points at k=6, +7.5 at k=8, +3.9 at k=16, +1.9 at k=24 and only +0.3 at k=64. Mid-sized models first generalise well and then deteriorate as they memorise the noise (the classical early-stopping picture), and this penalty shrinks as the model grows. This is the over-parameterised side of double descent seen along the training-time axis, and it is what the model-wise curve hides because only end-of-training numbers are plotted there.

Tree-wise (8k images, 15% noise, fixed node width W, 800 optimiser steps per node), all trees interpolate (100%), and test accuracy is 42.1% (W=0, 437 nodes, 0.23M parameters), 41.3% (W=8), 42.6% (W=32), 44.5% (W=128, 11 nodes) and 47.0% (W=512, a single node, i.e. just a wide MLP). Smaller nodes mean deeper, bigger trees and slightly worse accuracy, so the more of the memorisation work is done by one wide network instead of many repairs, the better the generalisation. A caveat: the single wide MLP of width 128 (67k parameters, 3000-step budget in the baseline vs 800 in the tree) interpolates with a smaller parameter count than any interpolating tree, so the tree is not a size-efficient way to interpolate. The two budgets differ, so this comparison is not perfectly controlled.

Why does the tree generalise worse than its root? Under noise, the root has 63.5% test accuracy and the full tree 58.6% (CIFAR-10); on Cats-vs-Dogs clean 75.8% to 72.0%. The children are trained only to fix training errors, many of which are flipped labels or hard outliers, and their closed-form weights are large (they must override the parent's logit gap). Any test point that happens to trigger a child is pushed towards an idiosyncratic class. The tree adds capacity exactly where the data is unreliable. On clean CIFAR-10 the damage is negligible (70.5% to 70.4%) because there are only 10 small repair nodes.

Receptive fields

Gradient (SmoothGrad) receptive fields of root class logits and of repair nodes are diffuse: the 50% gradient mass spreads over about 39-45% of the pixels, normalised entropy is 0.99+ for all nodes, and the centre-of-mass offset is small (0.01-0.04 of the half-diagonal), i.e. objects are centred and the nodes look at the whole image with a slight bias to the centre. There is no clear trend with depth: root nodes and depth-1, 2 and 3 repair nodes have statistically similar spread. The activation-maximisation images are high-frequency, adversarial-looking textures; only a few show structure (vertical stripes for deer and horse, diagonal stripes for frog), so they are weak evidence about what a node 'looks for'. The top-activating real images, in contrast, share strong background and colour cues (blue sky for airplanes, green grass for deer, frogs and birds), which suggests that, with a frozen low-capacity trunk, nodes lean on global colour and background statistics rather

than object parts. Repair nodes do not seem more local than the roots; they look at the same global features but in a slightly different direction of feature space, which is what lets them carve out individual samples. Note: figure titles of depth>=2 nodes in the existing RF pages carried a wrong class label (fixed in the code, requires re-running visualize_rf.py); the statistics are unaffected.

Size minimisation

The first lever is the per-node minimum-width search (linear, 16, 64, 256 hidden units): it keeps most nodes as small as possible, and on Cats-vs-Dogs noisy 15 of 22 nodes are plain linear classifiers. The second lever is pruning children with few positives. On clean CIFAR-10, removing children with fewer than 200 positives cuts parameters from 553k to 430k (-22%) while train accuracy drops 100% to 99.6% and test accuracy stays 70.4-70.5%. On Cats-vs-Dogs, pruning from 15 to 5 nodes shrinks 703k to 560k parameters with no loss at all, and for the noisy tree 584k to 560k (22 to 5 nodes) with 0.1 points less training accuracy and the same test accuracy. Most of the remaining parameters sit in the few large nodes (the root and the wide children), so further savings must come from them. Magnitude sparsification without fine-tuning is fragile: 50% sparsity loses 2.6 points of train accuracy, 80% sparsity collapses to about 53-66%, because the closed-form weights assume that the children are perfect. Therefore: pruning is effective, sparsification needs retraining. (The "full" baseline train accuracy in minimize.json is 99.99% rather than 100% because the cached features were float16; the code now stores float32.)

Repair depth as a noise detector

The repair depth (how deep in the tree a sample needed a repair) is an informative signal of label noise on CIFAR-10: AUROC 0.75, with 7% of root-correct samples being noisy versus 45% at depth 1. On Cats-vs-Dogs noisy the AUROC is only 0.56, because the binary problem is hard on clean data too (many clean hard examples get repaired as well).

Conclusions

The tree construction does what it promises: interpolation is guaranteed up to the depth cap, and the nodes can be very small. But interpolation by repair is not a good route to generalisation: the repair nodes fit noise and hard examples, test accuracy drops (by up to 5 points) when labels are noisy, and a single wide MLP interpolates with fewer parameters than any tree we grew. Double descent appears here mainly as epoch-wise overfitting that fades with width and as a monotone second descent in the model-wise curve; a clear sharp peak was not observed with our single seed and 15k training images. Limitations: single seed everywhere; frozen trunk trained for only 3 epochs on noisy labels; Cats-vs-Dogs trees truncated by depth 8; the tree-wise comparison uses unequal step budgets and 8k images; receptive-field analysis is qualitative on a limited number of nodes. Natural next steps are a final fine-tuning of the weights (instead of closed-form), multiple seeds, and intervals around the double-descent curve.

6. Future directions

• **Joint training.** The trunk is frozen. Fine-tuning the trunk together with the tree (hard step replaced by a straight-through estimator for the child outputs) could let the features adapt to the repair structure.

• **Calibrated, not closed-form, weights.** Closed-form weights assume a child is perfect. A child that fires wrongly on a test sample gets a large weight; learning the weights with a margin/hinge objective (eqs. 6–9 of the paper) or using soft gates should improve generalisation.

• **Tree as a label-noise detector.** Repair depth flags mislabelled training samples (section 2). Using it to discard or relabel samples, then regrowing, is a natural data-cleaning loop.

• **Smaller multiclass trees.** Share child networks across classes (a single multi-output repair network per node), use error-correcting output codes, or merge children with similar receptive fields.

• **Compression with fine-tuning.** Iterative magnitude pruning or L0 regularisation of node weights with re-optimisation instead of post-hoc pruning; node-width search with a global parameter budget.

• **Per-node features and feature selection.** Give each subtree its own trunk or an L1/zero-norm selected subset of features (open question 5 of the original note).

• **Theory.** Bound the number of nodes by margin / separability of the masked problems, relate depth to the complexity of the label noise, and explain the tree-wise test-error curve with the same tools as double descent.

7. Reproducibility

Private repository: https://github.com/ronakraichandani/MLOAssignment_2026EEY7568_Ronak. Scripts: run_tree.py (growth, interpolation), double_descent.py, tree_sweep.py, minimize.py, visualize_rf.py, make_report.py (this PDF). The Colab notebook notebooks/colab_run.ipynb runs all experiments in order. Seeds and arguments are stored in each results.json.

References: [1] Martinelli et al., A pyramidal delayed perceptron, IEEE Trans. Circuits Syst. 1990. [2] Weston et al., Use of the zero-norm with linear models and kernel methods, JMLR 2003. [3] Jayadeva, Deb, Chandra, Binary classification by SVM based tree type neural network, IJCNN 2002. [4] Nakkiran et al., Deep double descent, ICLR 2020. [5] Jayadeva, A tree type neural network (course note).